Cluster Computing (2026) 29:219
https://doi.org/10.1007/s10586-026-06025-9

Enhanced task distribution and adaptive resource management utilizing spike-induced graph neural networks with optimized offloading mechanisms in fog computing for improved efficiency

S. V. Juno Bella Gracia[1] · S. Srinivasan[2]

Received: 10 March 2025 / Revised: 31 December 2025 / Accepted: 8 February 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Increasingly, fog computing is being used by Internet of Things (IoT) applications to do computations at the network edge of devices instead of cloud computing. The low latency of fog computing, as opposed to the high latency of the cloud, delay-sensitive applications can benefit from improved quality of service. To increase service quality and reaction time in IoT applications, workloads dispersed efficiently between fog nodes. Therefore, this research proposes a novel model of Enhanced Task Distribution and Adaptive Resource Management utilizing Spike-Induced Graph Neural Networks with Optimized Offloading Mechanisms in Fog Computing for Improved Efficiency (TDARM-SIGNN-FC). To predict the optimal fog node for task offloading, task attributes and fog node characteristics are analyzed to ensure efficient resource management. Here, fog node prediction is performed using Spike-Induced Graph Neural Networks (SIGNN) to determine which node is best suited for offloading. The predicted node is then fed into the Starfish Optimization Algorithm (SFOA), which efficiently manages the task offloading process. Then the proposed TDARM-SIGNN-FC is implemented and the performance metrics like Energy Consumption (EC), Total Network Usage (TNU), Execution Time (ET), Average Response Time (ART), Standard Deviation (SD) and Cost are analyzed. Finally the performance of proposed TDARM-SIGNN-FC technique provides 24.65%, 21.95% and 29.56% lower energy consumption, 32.05%, 23.99% and 19.56% lower total network usage and 24.78%, 28.97% and 29.45% lower execution time while compared with existing methods like Prioritized Task offloading mechanism in Cloud-Fog Computing using improved Asynchronous Advantage Actor Critic Algorithm (PTOM-CFC-ACA), Hybrid Task scheduling technique in fog computing using fuzzy logic and Deep Reinforcement learning (TST-FC-DLR), and energy-efficient task offloading in fog computing for 5G cellular network (ETO-FC-5GCN) respectively.

Keywords Fog computing · Offloading mechanisms · Resource management · Spike-induced graph neural networks · Starfish optimization algorithm · Task distribution

List of symbols

$(T_1, T_2, \ldots, T_m)$	Dispersed sensors
μ_i	Data at a rate
$gh_1, gh_2, \ldots, gh_n$	Fog nodes
L_{ij}	The network latency between T_i and gh_j
$Esize_i/CX_l$	Data transfer time
$Esize_i$	The data size generated from T_i
T_i	Sensor
CX_l	Local network bandwidth
gh_j	Fog node
L_{jd}	Cloud latency between gh_j and the cloud
CX_d	Cloud network bandwidth
$CommCost_{ij}$	The entire sensor T_i communication cost
$P/P/1$	Queuing model
ϕ_{ij}	The service rate
P	The total number of sensor
ϕ_{jd}	The rate of service at fog node gh_j

✉ S. V. Juno Bella Gracia
bellagodwin.it@sairam.edu.in

[1] Department of Information Technology, Sri Sairam Engineering College, Chennai, Tamil Nadu, India

[2] Department of Computer Science and Engineering, R.M.D Engineering College, Kavaraipettai, Chennai, Tamil Nadu, India

Published online: 30 March 2026

TR_{ij}	The service time of task offloading
TR_{jd}	Service time for numerous sensors task
d	Particular application class
μ_{jd}	Total arrival rate of data from sensors related to a specific application d.
V_j	The offloading tasks from all sensors T_i
V_{jd}	Offloading data size
S_{ij}	Average service time
S_j	The response time of the fog
S_{jd}	The efficient task offloading
$S_{averade}$	The average fog nodes
$load_j$	The load function of offloading
f	The function representing the relationship between the input variables and the total cost.
$P_{t\times i}$	Transmission power
Pc	Circuit consumption power
E_{tr}	Mode transition energy
P_i	Power level task
MST	The maximum simulation time
TL, N	The total latency, the population size
TS	The total size
E_T	Timestamp when data transmission is completed
S_T	Timestamp when data transmission is begins
y_i	The value of the ith point
$\overline{y}$	The mean value
n	The count of data points
$Total\,\mathrm{Cost}$	The overall cost incurred
$y_1, y_2, \ldots\ldots, y_n$	The input variable
$\tilde{W}$	The system receiving a temporal graph of the fog network in the input layer.
t	Different time intervals to capture temporal evolution.
W	The graph snapshot
λ	The temporal graph representing the fog node prediction
W_{reset}	The reset of the graph snapshot
J	The temporal graph learning in fog computing
T	The propagation of information across nodes in spike-induced graph aggregation
G_b	The spike-based activations for fog node prediction
W_{th}	Threshold parameter for the spiking mechanism
$\tilde{g}$	The spiking message passing that propagates information through nodes
v	The neighborhood aggregation updates for a node in the graph convolution layer
k	The fog node state
δ	The node intensity
g	The task attributes based prediction
N	The total number of fog nodes
X	The dynamic node decisions for offloading
C	The ideal for resource-constrained fog nodes.
w	The efficient task in the spike propagation layer
$\tilde{A}$	The suitable fog node for the offloading task
$\otimes$	The fog node operator
Q	The energy constraint of fog node prediction
T	The propagation of information across nodes in spike-induced graph aggregation
ω	The fog node for the offloading task using the output layer
S_{ij}	The jth dimension position of the ith starfish
r	An arbitrary number between (0,1)
L_j	The lower bound of design variables in the jth dimension
U_j	The upper bound of design variables in the jth dimension
$Z^T_{i,p}$	The starfish's current position
$S^T_{i,p}$	The starfish's acquired position
$S^T_{best,\,p}$	The p dimension of the current location
P	The current iteration
$P_{\max}$	The maximum iteration
$L_{b.p} and\, U_{b,p}$	Bounds of design variables for the updated dimension P
$S^R_{k1,\,p} and\, S^R_{i,p}$	P Dimensional positions from two arbitrary particular starfish
$B_1\, and\, B_2$	Two arbitrary numbers between (-1, 1)
F_t	Energy of the starfish
R	Current iteration count
$R_{\max}$	Maximum iterative count
d_g	The five measured distances between other starfish and the world's best
gp	The five arbitrarily selected starfish
$r_1\, and\, r_2$	Arbitrary numbers between (0, 1).
$dg_1\, and\, dg_2$	Arbitrary selection in dg

1 Introduction

The Internet of Things (IoT) is gaining popularity in many aspects of daily life, including emergency response, smart homes, medical care and industrial automation [1]. The projected 75 billion IoT devices by 2025 would generate massive volumes of data, known as big data [2]. Big data must be saved, transported and evaluated to provide useful information for users. Unfortunately, most IoT end devices have limited storage and computational capabilities, making it impossible to store and process data [3–6]. Cloud computing offers infinite storage and processing capacity through powerful virtual servers is making it ideal for

non-time-sensitive applications with low response requirements [7]. Cloud computing's centralized nature makes it unsuitable for real-time processing applications because of significant delays and excessive network bandwidth use [8]. Cloud computing's centralized approach causes delays that are unacceptable for many IoT applications, such as VR, AR, driverless vehicles, industrial IoT and e-health [9–11]. Fog computing is a potential solution for shifting storage, networking, and processing capabilities to the network's edge, meeting the high computational needs and latency requirements of these applications [12]. Offloading IoT tasks to fog computing minimizes network congestion and latency, resulting in faster data analysis responses [13]. At the network's edge, nearer to IoT devices, edge computing enables reduced latency and quicker reaction times [14]. Fog and edge computing are not the same thing, though [15–17]. Through network devices like routers and gateways within their local area network (LAN), fog computing allows Internet of Things devices to access computing, networking, and storage services. Edge computing uses small data centers near IoT devices to deliver computing and storage services via Wi-Fi access points [18]. Quality of service (QoS) for intelligent applications such as autonomous driving, traffic control, and healthcare is made possible by fog computing [19]. Meeting QoS requirements and guaranteeing prompt response times depend on choosing the appropriate fog nodes for load balancing and job offloading [20].

The fog computing deployment presents significant challenges because offloading requires consideration of factors like latency, load distribution, privacy, and storage to decide whether to process tasks locally or offload them to the fog or cloud [21–24]. The complexity stems from the heterogeneity of fog devices, each with a varying computational capacity making efficient workload distribution challenging. Also, as request volumes increase, powerful fog nodes experience longer task queues leading to increase waiting times that exceed the latency requirements of time sensitive impacting performance and service reliability.

To overcome these issues, using enhanced task distribution and adaptive resource management utilizing SIGNN with optimal offloading mechanisms in fog computing effectively balance workloads across heterogeneous devices. This technique seeks to reduce latency, improve resource utilization and increase task execution efficiency in dynamic fog environment.

The proposed framework introduces Enhanced Task Distribution and Adaptive Resource Management in fog computing by integrating SIGNN with task offloading mechanisms optimized via the SFOA. Unlike conventional methods, this approach dynamically captures task dependencies and network topology, enabling efficient resource allocation, reduced latency, and improved computational efficiency. The synergy between SIGNN and SFOA ensures adaptive, scalable, and real-time task management, addressing both spatial and temporal resource constraints in heterogeneous fog environments, which existing solutions fail to handle effectively.

Major contribution of this paper includes,

- Introduction of TDARM-SIGNN-FC, a model integrating SIGNN with optimized offloading mechanisms for enhanced task distribution and adaptive resource management in fog computing.
- SIGNN analyzes task attributes and fog node characteristics to accurately predict the most suitable fog node for offloading.
- The SFOA efficiently manages the offloading process, balancing computational load across fog nodes.
- The model reduces energy consumption, execution time, and network usage through intelligent task allocation and adaptive management.
- TDARM-SIGNN-FC demonstrates high scalability and practical applicability for real-world IoT-enabled fog computing environments.

Following manuscript is arranged as below: Sect. 2 defines literature survey, Sect. 3 explains proposed approach, Sect. 4 demonstrates outcomes and discussion, Sect. 5 concludes the manuscript.

2 Literature survey

Several research works presented in the literatures were based enhanced task distribution and adaptive resource management utilizing deep learning; few of them are reviewed here,

Choppara and Mangalampalli [25] have presented Fuzzy logic and deep reinforcement learning were used in a hybrid task scheduling technique in fog computing. Here, a Takagi–Sugeno fuzzy inference system was combined with fuzzy logic and deep reinforcement learning (HTSFFDRL) to create a hybrid task scheduling method in fog computing. The hybrid technique enables dynamic task prioritization and real-time scheduling rule changes by continuously engaging with the environment. It provides lower energy consumption and it provides higher cost.

Mangalampalli et.al. [26] have presented enhanced Asynchronous Advantage Actor Critic Algorithm for Cloud-Fog Computing's prioritized task offloading mechanism. Prioritizing tasks and determining whether to offload them to fog or cloud nodes were decisions made by the task offloading mechanism (PTOMCFIA3C). In order to optimize scheduling and speed up learning by offloading work to fog/cloud

nodes, the enhanced Asynchronous Advantage Actor Critic method a deep reinforcement learning method was used to model PTOMCFIA3C. It provides lower average response time and it provides highest total network usage.

Muhamad et.al. [27] have presented energy-efficient fog computing task offloading for 5G cellular networks. Problems like excessive delay, higher energy consumption, and heavy load have been brought about by the expansion of IoT devices. An innovative low-latency and energy-efficient fog computing job offloading method was the Gale Shapley Stable Marriage Matching algorithm. Gale Shapley Stable Marriage matching game pairs four mobile devices with four fog nodes for one-on-one matching.. It provides standard deviation and it provides highest execution time.

Umer et.al. [28] have presented task offloading and scheduling model for IoT logistics that was fault tolerant and prioritized. A new fault-tolerant, priority-based task offloading and scheduling paradigm for Internet of Things logistics called FP-TOSMwas presented The Analytic Hierarchy Process (AHP) ranked IoT tasks based on a number of parameters. Tasks within that range were offloaded to a fog node, while those below that barrier were carried out locally. It provides lowest execution time and it provides highest standard deviation.

Table 1 Comparison of literature survey

Author (s)	Models	Objectives	Advantages	Disadvantages
Choppara, P. and Mangalampalli, S.S., (2024) [25]	Fuzzy Logic, DRL	Hybrid task scheduling in fog computing	Better resource use, lower latency	High computation cost
Mangalampalli, S., et.al (2024) [26]	Improved A3C	Prioritized task offloading	Efficient distribution, better decisions	High training cost
Muhamad, W.N.W., et.al (2024) [27]	Energy-efficient offloading	Optimize 5G task offloading	Lower energy use, better QoS	Limited adaptability
Umer, A., et.al (2025) [28]	Fault-tolerant, priority-based scheduling	IoT logistics task management	More reliable, prioritized tasks	Computation overhead
Wei, Z., et.al (2024) [29]	Privacy-Preserving RL	Secure vehicular fog computing	Strong privacy, efficient learning	High computation
Zhang, S.H., et.al (2025) [30]	Snake Optimizer	IoT task offloading	Better resource use, high optimization	Complex process
Choppara, P. and Mangalampalli, S., (2025) [31]	DRL	Cloud-fog task scheduling	Faster execution, better efficiency	Long training time

Wei et.al. [29] have presented privacy-Preserving Hierarchical Reinforcement Learning Framework for Task Offloading in Low-Altitude Vehicular Fog Computing.Here, offer a hierarchical reinforcement learning framework to support learning-based approaches. The framework has two levels: the local level offers Deep Reinforcement Learning (DRL) models for task vehicles and UAVs, and the cross-regional level coordinates local experiences. To developed an Attention-enhanced Federated Proximal Policy Optimization (AFedPPO) approach for decentralized job offloading in low-altitude VFC systems. It provides lower energy consumption and it provides highest execution time.

Zhang et.al. [30] have presented Multi-task learning in fog computing based on the Internet of Things employs a multi-strategy fusion snake optimizer for task offloading and scheduling. Here, a circle map was used to initialize the population's positions in order to improve their traversal and non-repetitive properties, which in turn improves the algorithm's accuracy and rate of convergence. An international effort to increase population diversity uses the Synergistic Optimization Strategy. It provides lower total network usage and it provides higher energy consumption.

Choppara and Mangalampalli [31] have presented an efficient deep reinforcement learning based task scheduler in cloud-fog environment. Here, DRLMOTS (Deep Reinforcement Learning-based Multi-Objective Task Scheduler in Cloud Fog Environment) was used to integrate fog and cloud environments. The scheduler dynamically allocates compute to fog nodes or cloud resources depending on task criteria, including length and capacity. The approach simulates both randomized and actual Google Jobs workloads using a Deep Q-Learning Network model. It provides lower average response time and it provides higher standard deviation. Table 1 shows the comparison of literature survey,

Several studies have addressed task offloading and resource management in fog computing using various approaches. Choppara and Mangalampalli [25] employed a hybrid fuzzy logic and deep reinforcement learning method (HTSFFDRL) for dynamic task scheduling, achieving lower energy consumption but limited execution efficiency. Mangalampalli et al. [26] proposed PTOM-CFC-ACA for prioritized task offloading, improving response time but increasing network usage. Muhamad et al. [27] utilized Gale-Shapley matching for energy-efficient 5G fog computing, while Umer et al. [28] and Wei et al. [29] explored

fault-tolerant and hierarchical reinforcement learning-based offloading, respectively. Zhang et al. [30] applied multi-strategy optimization, improving network usage but increasing energy consumption. Despite these advances, existing methods often struggle with simultaneous optimization of task distribution, fog node prediction, and adaptive offloading in dynamic IoT environments. To address these limitations, the proposed TDARM-SIGNN-FC integrates SIGNN for accurate node prediction with the SFOA for efficient task offloading, significantly improving energy efficiency, execution time, and overall scalability in fog computing.

3 Proposed methodology

In this section, TDARM-SIGNN-FC approach for effectual task allocation and resource management in fog computing, utilizing SIGNN for fog node prediction and SFOA for optimized task offloading. This process includes three steps: fog system model, fog node prediction and task offloading. SIGNN is used to predict the suitable fog node for offloading by utilizing the fog network and collecting inters node relationships. The predicted node is then given into SFOA, which dynamically improves the task distribution process

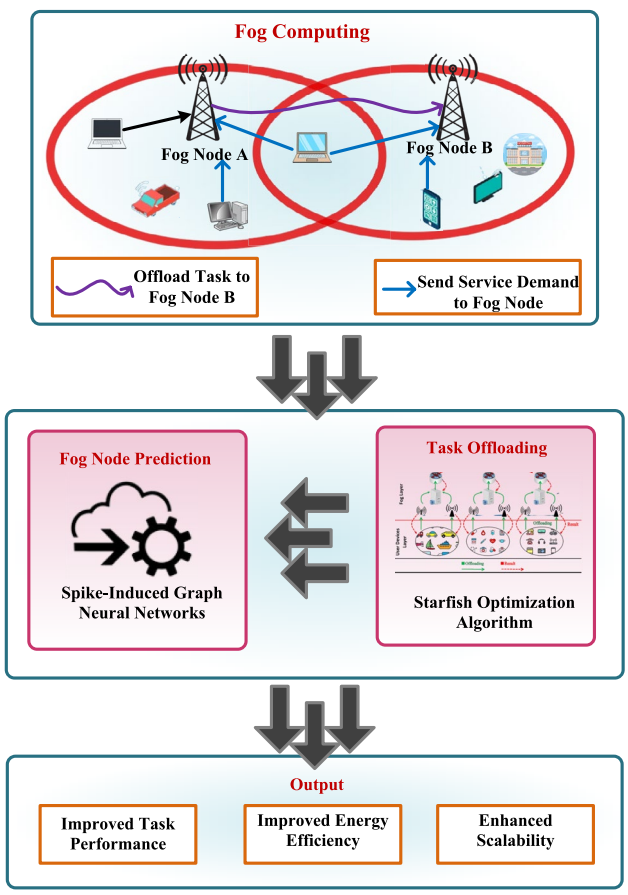

Fig. 1 Block diagram for proposed of TDARM-SIGNN-FC method

to maximize resource efficiency. The proposed TDARM-SIGNN-FC approach's block diagram is shown in Fig. 1. Consequently, the following provides a thorough explanation of each step,

3.1 Fog system model

To maximize task offloading in fog computing infrastructure, the SIGNN meta-heuristic is applied in this section. The primary goal is to reduce task response times while taking network bandwidth, latency, and the current load on the fog devices into consideration.

i. Fog network architecture

Three layers make up this IoT-fog network architecture: cloud, fog, and IoT. The IoT layer, which is at the bottom, uses a local network to link geographically dispersed sensor nodes. The IoT layer collects data from a variety of applications, including traffic intensity, weather, and air quality. IoT layer is linked to fog layer, which is made up of many fog nodes. The gathered sensor data, including sensor data arrival and service rates, is aggregated, filtered, and light processed by each fog node. Requests to offload tasks are sent by sensors to fog nodes, which then forward them to a master node. The offloaded tasks are scheduled by the master node using information gathered from fog agents. Finally, the cloud layer offers significant processing and storage capacity. To connect to the cloud layer, the fog layer uses the Internet rather than the local network. The cloud layer handles the processing and storing of massive amounts of data. Figure 2 shows the fog computing architecture,

ii. *Fog model* formulation

The IoT layer is made up of dispersed sensors $(T_1, T_2, \ldots, T_m)$ with each sensor producing data at a rate μ_i. The fog layer is made up of many fog nodes $gh_1, gh_2, \ldots, gh_n$. The sum of network latency L_{ij} and data transfer time $Esize_i/CX_l$, where $Esize_i$ the size of sensor T_i data and CX_l is the local network bandwidth.The cost of data transmission between sensor T_i and fog node gh_j is given in Eq. (1).

$$FogDCost_{ij} = L_{ij} + \frac{Esize_i}{CX_l} \tag{1}$$

here, L_{ij} represents the network latency between T_i and gh_j; $Esize_i$ indicates the data size generated from T_i and CX_l represents the local network bandwidth. The cloud latency L_{jd} and data transfer time gh_j among fog nodes and the cloud are combined to calculate $Esize_i/CX_l$ where CX_d

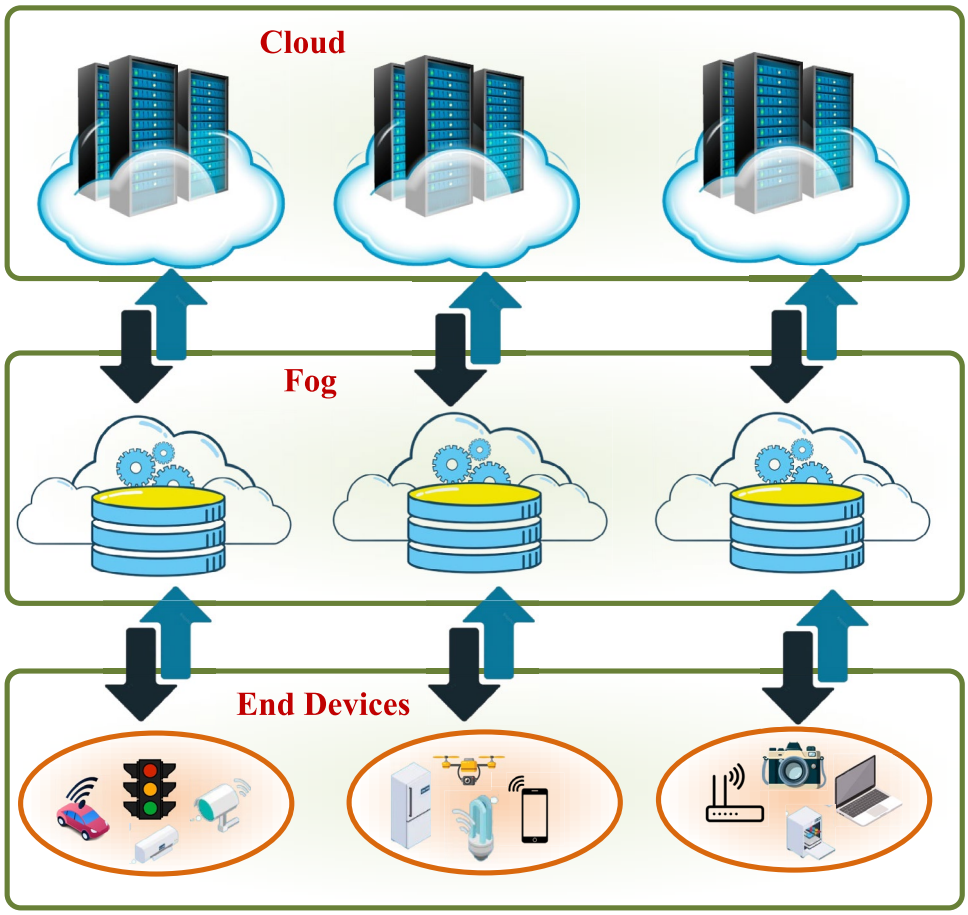

Fig. 2 Architecture of fog computing

represents the cloud network bandwidth. The cost of communication between the cloud and fog node gh_j is expressed in Eq. (2).

$$CloudDCost_{ij} = L_{jd} + \frac{Esize_i}{CX_d} \tag{2}$$

where, L_{jd} indicates the cloud latency between gh_j and the cloud and CX_d represents the cloud network bandwidth. The total communication cost for sensor T_i offloading. Expressed in Eq. (3).

$$CommCost_{ij} = CloudDCost_{ij} + FogDCost_{ij} \tag{3}$$

here, $CommCost_{ij}$ represents the entire sensor T_i communication cost. At a rate of μ_i, each sensor T_i produces data according to a Poisson distribution. The fog nodes receive the created data and use it for basic analysis, aggregation and filtering. The fog nodes then transmit the information to the cloud for more in-depth analysis, expensive processing, and storage. Equation (4) is used in the $P/P/1$ queuing model to determine the service time of task offloading (TR_{ij}) of sensor T_i to fog nodes gh_j.

$$TR_{ij} = \frac{1}{\phi_{ij} - \mu_i} \tag{4}$$

here, ϕ_{ij} represents the service rate; P indicates the total number of sensor and μ_i denotes the sensor T_i's data arrival. Equation (5) is utilized to determine the service time for numerous sensors task (TR_{jd}) that are assigned to a particular application class d.

$$TR_{jd} = \frac{1}{\phi_{jd} - \mu_{jd}} \tag{5}$$

where ϕ_{jd} stands for the rate of service at fog node gh_j for a particular application class $d\, and\, \mu_{jd}$ is calculated using Eq. (6) and (μ_{jd}) represents total arrival rate of data from sensors related to a specific application d.

$$\mu_{jd} = \sum_{i \in d} \mu_{ji} \tag{6}$$

The usage of fog nodes (gh_j, V_j) is the aggregate of offloaded tasks from all sensors T_i and it is given in Eq. (7).

$$V_j = \sum_d V_{jd} = \sum_d \sum_{i \in d} y_{ij} \times \frac{\mu_{ji}}{\varphi_{ij}} \tag{7}$$

here, V_j denotes the offloading tasks from all sensors T_i; The offloading data size is denoted by V_{jd}, and if sensor T_i tasks are offloaded to fog node gh_j, then $y_{ij} = 1$. As a consequence of task offloading, the overall reaction time of the sensor T_i workload is determined by the total cost of fog-cloud communication, IoT-fog communication, and typical duration of service for the fog nodes given in Eq. (8).

$$\begin{aligned} S_{ij} &= Comm\text{Cost}_{ij} + TR_{jd} \\ &= L_{jd} + \frac{Esize_i}{CX_d} + L_{ij} + \frac{Esize_i}{CX_l} + \frac{1}{\phi_{jd} - \sum_{i \in d} \mu_{ji}} \end{aligned} \tag{8}$$

where, S_{ij} represents average service time. The mean reaction time of fog node gh_j after all task have been offloaded in Eq. (9).

$$S_j = \sum_d \frac{S_{jd}}{1 - V_{jd}} \tag{9}$$

here, S_j indicates the response time of the fog and S_{jd} represents the efficient task offloading. In comparison to all fog nodes, the load $load_j$ on fog node gh_j is given in Eq. (10).

$$load_j = 1 - \frac{S_j - S_{averade}}{\sum_j S_j} \tag{10}$$

here, $S_{averade}$ represents the average fog nodes and $load_j$ indicates the load function of offloading.

3.2 Fog node prediction using spike-induced graph neural networks

In this section, Spike Induced Graph Neural Networks (SIGNN) [32] is discussed to predict the most suitable fog node for offloading the task. The node is predicted related to the task attributes and fog nodes characteristics. The prediction is related to which node handle the task most efficiently, considering factors like energy consumption, latency, and node availability. Based on factors including workload, energy consumption and network congestion, SIGNN models the fog nodes and their connections as a graph. The spike induced activation functions aid in choosing the best node for offloading. The Fig. 3 illustrates the architecture of SIGNN,

Figure 3 displays the SIGNN architecture. The system for temporal graph learning that makes use of spike-induced graph aggregation and multiple time granularities (MTG). The first step is MTG aware graph sampling, which uses neighborhood sampling with K-hop selection to create a series of graph snapshots that record both continuous and binary spike features over time. Spike-induced graph aggregation is used to process the sampled multiple snapshot sequences. An aggregator uses several algorithms, such as the SAGE updater and BLIF updater, to update features. In order to efficiently capture temporal dependencies, the architecture is composed of several aggregation-update layers across several channels. In order to produce robust node embedding and preserve temporal activation (TA) and feature dynamics for tasks that come after, the processed embedding are next subjected to temporal embedding aggregation, which makes use of temporal pooling and MTG aggregation. The input layer processes graph-structured data where nodes represent fog devices with attributes, while edges define network connections. It also incorporates task constraints and computation load for efficient offloading is given in Eq. (11).

$$\tilde{W}^t = W^{t-1} + \lambda \left(W_{reset} - W^{t-1} + J^t \right) \tag{11}$$

here, $\tilde{W}$ represents the system receives a temporal graph representing the fog network in the input layer; t is the different time intervals to capture temporal evolution; W denotes the graph snapshot; λ indicates the temporal graph representing the fog node prediction; W_{reset} indicates the reset the graph snapshot and J denotes the temporal graph learning in fog computing. The spiking mechanism allows SIGNN to learn and adapt to changes in the fog network, enhancing task allocation efficiency and expressed in Eq. (12).

$$T^t = G_b \left(\tilde{W}^t, W_{th}^{t-1} \right) \tag{12}$$

where, T represents the propagate information across nodes in spike induced graph aggregation and G_b denotes the spike based activations for fog node prediction. A SIGNN is used to depict the fog computing environment to predict the fog nodes with nodes standing for fog devices and edges for network communication. The network communication of fog node is given in Eq. (13).

$$\tilde{g}_v^{t,(k+1)} = \delta \left(g_{Nv}^{t,(k)} X_g^{(k)} + g_{Nv}^{t,(k)} C_g^{(k)} \right) \tag{13}$$

where, $\tilde{g}$ represents the spiking message passing propagates information through the nodes; v denotes the neighbourhood aggregation updates node in graph convolution layer; k indicates the fog node state; δ represents the node intensity; g denotes the task attributes based prediction; N indicates the total number of fog node; X represents the dynamic node decisions for offloading and C denotes the ideal for resource constrained fog node. The SIGNN model aggregates information from neighboring nodes using a

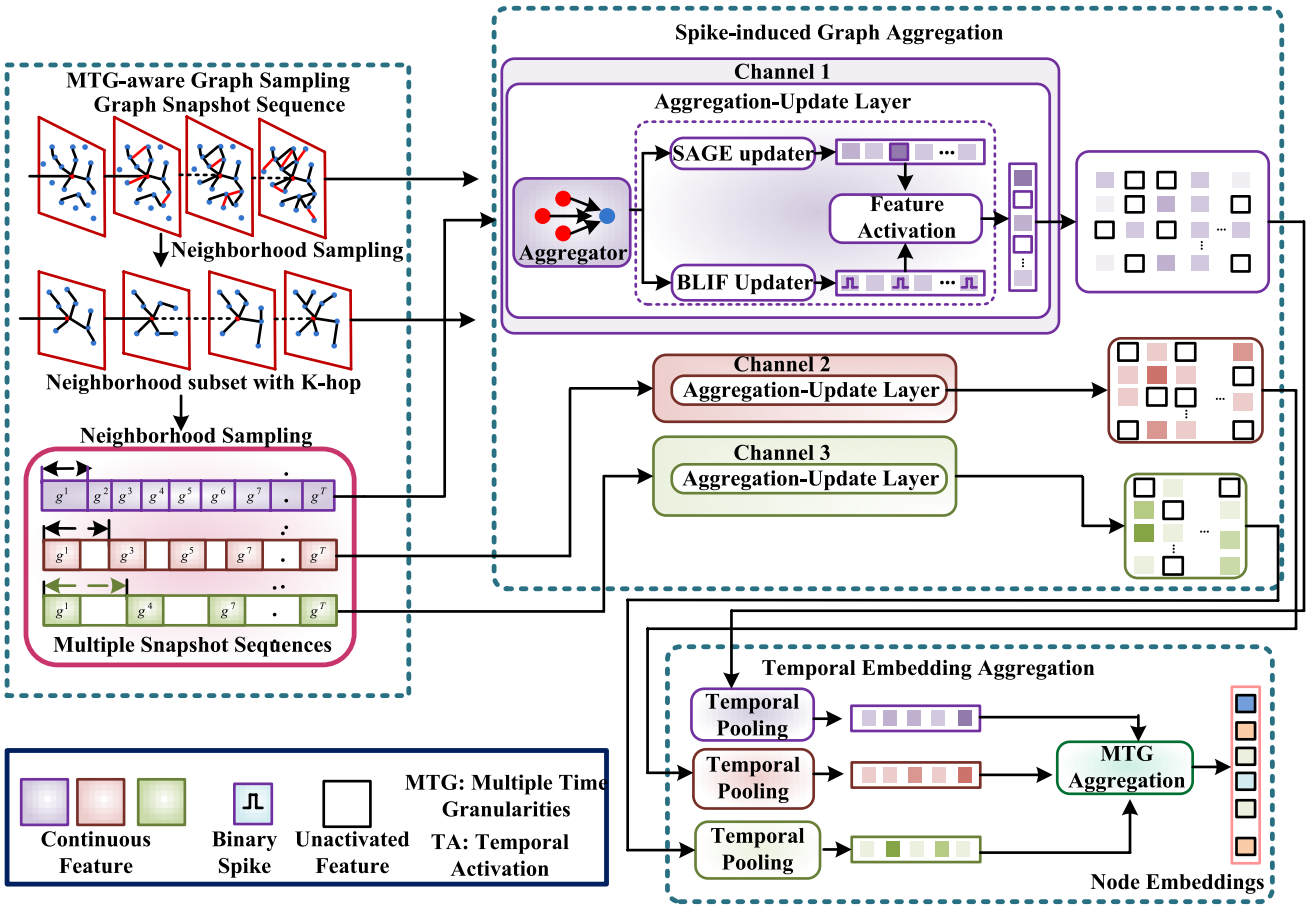

Fig. 3 Architecture diagram for SIGNN

spiking based graph convolution mechanism to infer optimal offloading decisions and given in Eq. (14).

$$w_v^1 = \left(\tilde{A}_v^1 \otimes Q_1\right) \cdot 1 \tag{14}$$

here, w represents the efficient task in spike propagation layer; $\tilde{A}$ indicates the suitable fog node for offloading task; $\otimes$ denotes the fog node operator and Q represents the energy constraint of fog node prediction. Spike based computing and graph neural network principles are combined to create SIGNN, which guarantees reliability and is very useful for intelligent fog node prediction. The intelligent task offloading for fog node prediction is expressed in Eq. (15).

$$W^t = \tilde{W}^t \left(1 - T^t\right) + W_{reset} T^t \tag{15}$$

where, T represents the propagate information across nodes in spike induced graph aggregation. The spiking mechanism allows SIGNN adapt to changes in the fog network enhancing task allocation efficiency for predict the fog node accurately. To predict the most suitable fog node for offloading task is given in Eq. (16).

$$W_{th}^t = \omega W_{th}^{t-1} + (1 - \omega)\, T^t \tag{16}$$

here, ω represents the fog node for offloading task using output layer. Finally, SIGNN predicted the most suitable fog node for offloading task. Then predicted node is fed into SFOA phase.

3.3 Task offloading using starfish optimization algorithm (SFOA)

In this sector, SFOA [21] is used to schedule and offload the task to the predicted fog node. The SFOA inspired by the adaptive behaviour of starfish or sea stars is utilized for task offloading in SIGNN to enhance computational efficiency in fog computing. SIGNN requires fast task scheduling and SFOA ensures the balancing workloads across fog nodes while minimizing latency, energy consumption and communication costs. By dynamically adapting to node availability, SFOA selects the best fog nodes for offloading to preventing ensuring smooth execution and bottlenecks. Its intelligent optimization method improves computing performance and making it well- suited for SIGNN complicated distributed processing demands. Here, a detailed process for using

SFOA to attain the appropriate SIGNN is explained. The entire step process is provided below,

Step 1: Initialization

In the SFOA initialization phase, starfish locations are created randomly among design variable boundaries derived in Eq. (17).

$$S = \begin{bmatrix} S_{11} & S_{12} & \dots & S_{1D} \\ S_{21} & S_{22} & \dots & S_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ S_{N1} & S_{N2} & \dots & S_{ND} \end{bmatrix}_{N \times D} \tag{17}$$

here, S implies matrix used to store the starfish positions, which has a size of $N \times D$. The population size is N, and the design variable dimension is D. Each starfish's position during the initialization phase is given in Eq. (18).

$$S_{ij} = L_j + r\,(U_j - L_j)\,,\ i = 1, 2, \dots, N,\ \ j = 1, 2, \dots, D \tag{18}$$

here, S_{ij} signifies j^{th} dimension position of ith starfish; r implies random number between $(0, 1)$, L_j and U_j denotes lower bound as well as upper bound of design variables in the jth dimension, correspondingly. Fit values of each starfish are found by assessing from the objective function following the creation of the initialized matrix of positions expressed in Eq. (19).

$$I = \begin{bmatrix} I\,(S_1) \\ I\,(S_2) \\ \vdots \\ I\,(S_N) \end{bmatrix}_{N \times 1} \tag{19}$$

where, I stands for the $N \times 1$ matrix used to maintain and update the fitness values that have been obtained. The exploration and exploitation phases begin when SFOA enters the main loop following initialization.

Step 2: Random Generation

Create input parameter at random after initialization. It depends on a specific requirement for hyper parameters.

Step 3: Fitness Function

It uses initialized values to create anarbitrary solution. It is calculated by exploration node. This is given in Eq. (20).

$$Fitness\,Function = optimal\,[Task\,Offloading] \tag{20}$$

Step 4: Exploration Phase

To replicate the exploratory behaviour of starfish, incorporate eyes into the ends of each limb of the SFOA algorithm's exploration phase to simulate the search capabilities of the fish's five arms. This is expressed in Eq. (21).

$$\begin{cases} Z_{i,p}^T = S_{i,p}^T + b_1 \left(S_{best,\,p}^T - S_{i,p}^T\right) \cos\omega, & R \le 0.5 \\ Z_{i,p}^T = S_{i,p}^T - b_1 \left(S_{best,\,p}^T - S_{i,p}^T\right) \sin\omega & R > 0.5 \end{cases} \tag{21}$$

here, $Z_{i,p}^T\, and\, S_{i,p}^T$ represent the starfish's current and acquired positions; $S_{best,\,p}^T$ denotes the p dimension of the current location, where p represents one of five arbitrarily chosen dimensions from D. Here, $b_1\, and\, \omega$ are calculated in Eqs. (22) and (23).

$$b_1 = (2r - 1)\,\pi \tag{22}$$

$$\omega = \frac{\pi}{2} \cdot \frac{P}{P_{\max}} \tag{23}$$

where, $P_{\max}$ denotes the maximum iteration and P represents the current iteration. The arms would rather stay in the old position than move to the new one if the updated position is outside the bounds of the design variables. Equation (24) expresses the previously mentioned.

$$S_{i,p}^{R+1} = \begin{cases} Z_{i,p}^R & L_{b.p} \le S_{i,p}^R \le U_{b,p} \\ S_{i,p}^R & otherwise \end{cases} \tag{24}$$

where, $L_{b.p} and\, U_{b,p}$ denotes the bounds of design variables and p indicates the updated dimension. Here, a starfish uses the position data from other starfish to move only one arm in search of food source. The updated position is expressed in Eq. (25).

$$Z_{i,q}^R = F_t S_{i,p}^R + B_1\ \left(S_{k1,\,p}^R - S_{i,p}^R\right) + B_2\ \left(S_{k2,\,p}^R - S_{i,p}^R\right) \tag{25}$$

here, $S_{k1,\,p}^R\, and\, S_{i,p}^R$ represents the p dimensional positions from two arbitrarily particular starfish; $B_1\, and\, B_2$ indicates the two arbitrary numbers among $(-1, 1)$; p is the arbitrarily particular number in the D dimensions and F_t denotes energy of starfish, which is given in Eq. (26).

$$F_t = \frac{R_{\max} - R}{R_{\max}} \cos\omega \tag{26}$$

where, R indicates the current iteration and $R_{\max}$ represents the maximum iterative count. As with the previous updating rule, a starfish remain in its former location rather than travel to the updated one if its obtained position is outside the boundary. Figure 4 shows the flowchart of SFOA.

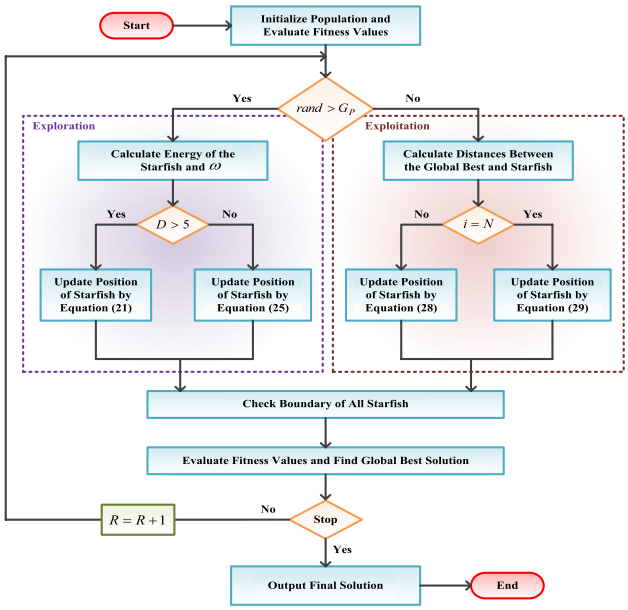

Fig. 4 Flow chart of SFOA for task offloading

Step 5: Exploitation Phase

Two update techniques uses the preying and regeneration behaviours are developed in the SFOA exploitation to find global solutions. Then, choose two distances at random as the acknowledgement to update each starfish's location. The distance is given in Eq. (27).

$$d_g = \left(S_{best}^R - S_{gp}^R\right), \; g = 1, \ldots\ldots, 5 \tag{27}$$

here, d_g denotes five measured distances among other starfish and the world's best and gp denotes the five arbitrarily selected starfish. This represents each starfish's updating rule in its predatory behavior is represented in Eq. (28).

Table 2 Hyperparameter of the proposed method

Hyperparameter	Value	Hyperparameter	Value
Number of layers	3	Population size	30
Number of hidden units per layer	128	Maximum iterations	50
Activation function	ReLU	Exploration factor	0.7
Batch size	64	Exploitation factor	0.3
Epochs	100	Learning rate	0.001

$$Z_i^R = S_i^R + r_1 dg_1 + r_2 dg_{21} \tag{28}$$

here, $r_1 \, and \, r_2$ is an arbitrary number between $(0, 1)$ and $dg_1 \, and \, dg_2$ indicates the arbitrary selection in dg. According to the parallel 2-directional search approach, certain starfish candidates are regressing within the similar iteration while others are heading toward the better directing answer. Model the updating rule and update the position during the regeneration phase in Eq. (29).

$$Z_i^R = \exp\left(-R \times N / R_{\max}\right) S_i^R \tag{29}$$

here,N denotes population size. A starfish will cut and lose an arm to get away from a predator. The position attained from the design variables boundaries of position is given in Eq. (30).

$$S_i^{R+1} = \begin{cases} Z_i^R & L_{b.} \leq S_i^R \leq U_b \\ L_b & Z_i^R < L_b \\ U_b & Z_i^R > U_b \end{cases} \tag{30}$$

The regeneration step involves only one function evaluation per iteration is significant for avoiding local solutions and enhancing global convergence. Pseudo-code of SFOA is represented in Algorithm 1

Table 3 Simulation parameters and system configuration

Simulation parameter	Value	Simulation parameter	Value
Number of fog Nodes	1500	fog node energy availability	300 J
Number of tasks	1000	Network bandwidth	50 Mbps
Task computational Load	500 CPU cycles	Network latency	25 ms
Task data size	25 MB	Fog node memory	16 GB
Task priority level	3	Packet loss rate	0.5%
Task deadline	250 ms	Hardware	Intel i7-10750H CPU, 16 GB RAM, NVIDIA RTX 3060 GPU
Fog node processing capacity	5 GHz	Software	Python 3.9, PyTorch 2.0, NetworkX 2.8, NumPy, Matplotlib

Algorithm 1 Pseudo-code of SFOA algorithm

Input: Algorithm parameters $(N, R_{\max}\ and\ G_p)$

Output: The global solution

```
Initialize input parameters, problem information, boundary, etc.
Initialize positions by Eq. (17) and assess the fitness values using Eq. (19)
While R < R_max Do
    If rand > G_p % Exploration phase
        Compute ω by Eq. (23) and energy F_t by Eq. (26)
        For each starfish do
            If D > 5% When the dimension is larger than 5
                Generate five randomly p dimensions among D
                For j = 1:5 Do % update five randomly selected p dimensions
                Compute b_1 by Eq. (22)
                Update the dimension of position in starfish using Eq. (21)
            End For
            Check the boundary of position in starfish using Eq. (21)
        Else % When the dimension is not larger than 5
            Create two random numbers B_1 and B_2
            Select a random dimension p
            Update the p dimension of position using Eq. (25)
            Check the boundary of starfish by Eq. (24)
        End If
```

```
        End For
Else % Exploitation phase
        Generate distances among the best position and starfish
        For each starfish do
                Generate random number r_k , and update the position utilizing Eq. (28)
                If i = N Do
                        Update the position utilizing Eq. (29) % regeneration phase
                End If
        Check the boundary of starfish by Eq. (30)
End If % Updating finished
Evaluate the fitness values for all starfish % Obtain fitness values
Attain the global best solution in iteration R step
R = R + 1
End While
Obtain global solution
```

Step 6: Termination

In this step, SFOA is completed and the best solution obtained through the each process iteration returned as output. The SFOA process has been finalized to ensure efficient task offloading. Then TDARM-SIGNN-FC assesses the Enhanced Task Distribution and Adaptive Resource Management with Optimized Offloading Mechanisms in Fog Computing by decreasing the energy consumption and execution time.

4 Result and discussion

The simulation outcomes of proposed TDARM-SIGNN-FC technique are discussed in this sector. The simulations are performed TensorFlow under the simulation category and PureEdgeSim, iFogSim and Pytorch simulations. For the evaluation of the proposed TDARM-SIGNN-FC framework, a synthetic dataset simulating IoT-enabled fog computing environments was created. The dataset contains tasks with attributes like computational load, data size, priority level, and deadline requirements. Fog nodes are characterized by processing capacity, memory, energy availability, and network connectivity. The dataset comprises 1000 tasks distributed across 1500 fog nodes for training and testing purposes. Data preprocessing included normalization of task attributes and fog node capacities to ensure uniform scaling. The dataset is divided into 70% for training and 30% for testing, ensuring diversity in task types and fog node characteristics. This setup allows the model to learn effective task-to-node mappings and evaluate performance metrics such as Energy Consumption, Total Network Usage, Execution Time, Average Response Time, Standard Deviation, Degree Imbalance and Cost under realistic workload scenarios. Obtained outcome of the proposed TDARM-SIGNN-FCmethod is analyzed with existing techniques including TST-FC-DLR [25], PTOM-CFC-ACA [26] and ETO-FC-5GCN [27] correspondingly. Table 2 shows the hyper parameter of the proposed method and Table 3 depicts the simulation parameters and system configuration.

4.1 Performance measures

Performance of proposed technique is evaluated utilizing Energy Consumption, Total Network Usage, Execution

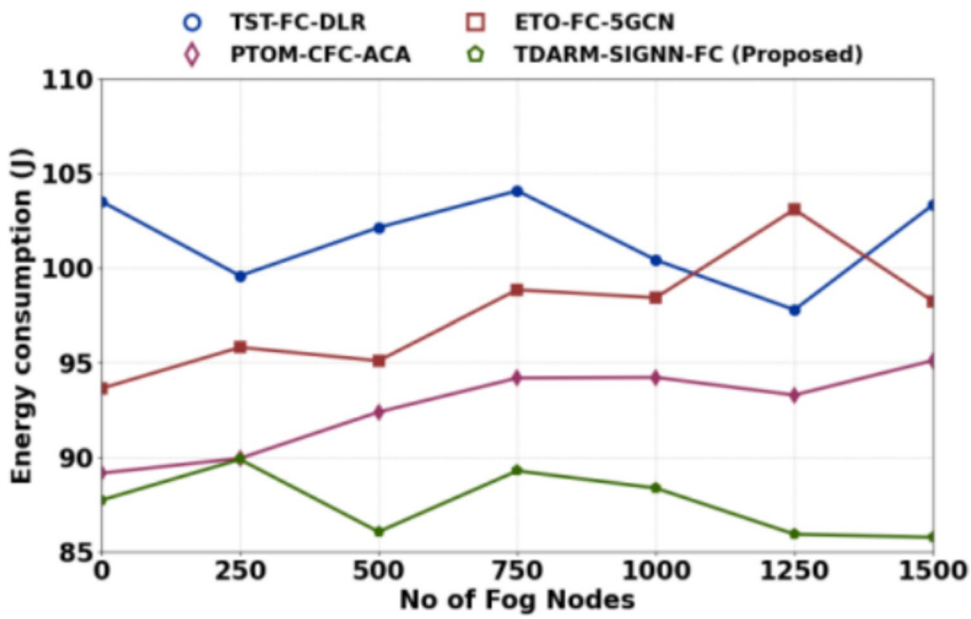

Fig. 5 Performance analysis of energy consumption

Table 4 Performance analysis of Total Network Usage

Method	Total network usage (bytes)					
	Number of fog nodes					
	250	500	750	1000	1250	1500
TST-FC-DLR	103.36	99.35	101.14	102.86	98.55	95.85
PTOM-CFC-ACA	96.85	96.21	98.62	99.71	99.68	98.68
ETO-FC-5GCN	91.63	93.80	93.09	96.83	96.41	101.08
TDARM-SIGNN-FC (Proposed)	90.40	92.22	88.18	91.18	90.16	87.80

Time, Average Response Time, Standard Deviation and Cost performance metrics.

4.1.1 Energy consumption

The amount of energy used by a system, piece of equipment, or procedure during a specified period of time is known as energy consumption. Understanding device power requirements, operating expenses, and environmental impact depend on evaluating computing and electronics energy consumption. This is computed using Eq. (31).

$$E_{consumption} = \sum_{i \in mi} \left(\left((1+\alpha) P_{t \times i} + Pc \right) \frac{L}{W \times b_i} + E_{tr} \right) + \sum_{j \in ti} P_i \frac{C}{F_i} \tag{31}$$

where, $P_{t \times i}$ denotes transmission power; Pc denotes circuit consumption power; E_{tr} represented as mode transition energy and P_i denoted as power level task.

4.1.2 Total network usage

Total Network Usage stands for the total count of data transmitted across network layers, including input, hidden, and output layers during model training and inference. It is calculated using Eq. (32).

$$TNU = MST / (TL \times TS) \tag{32}$$

here, TNU represents the total network usage; MST denotes the maximum simulation time; TL indicates the total latency and TS represents the total size.

4.1.3 Execution time

Execution time stands for the entire time taken by the model or system to process and analyze the data. Fog computing optimizes job allocation and resource management reducing execution time by avoiding delays and efficiently balancing computational loads. The execution time is calculated by Eq. (33).

$$Execution\ time = \frac{E_T}{S_T} \tag{33}$$

where, E_T denotes timestamp when data transmission is completed and S_T denotes timestamp when data transmission is begins.

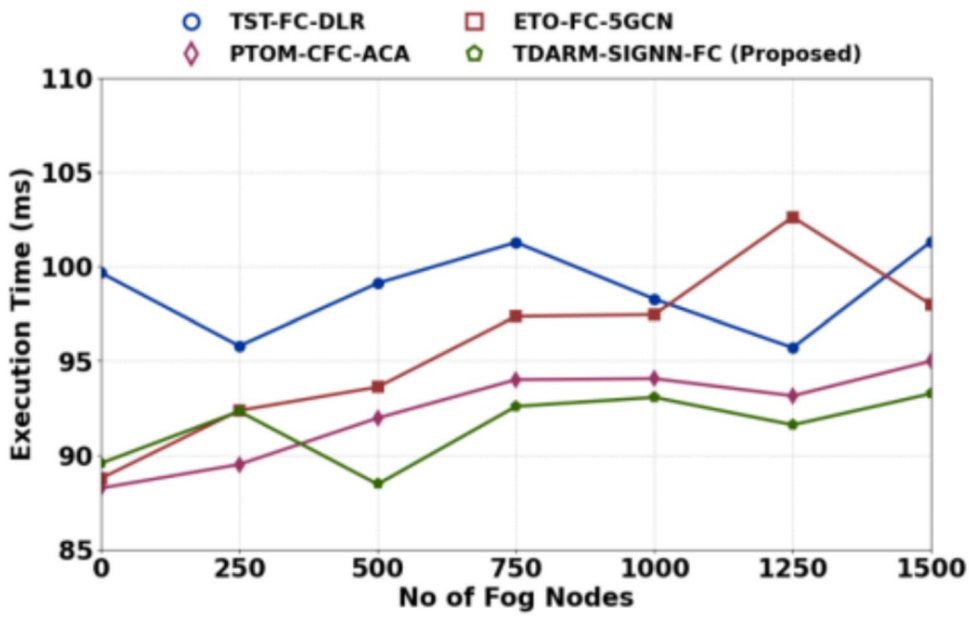

Fig. 6 Performance analysis of execution time

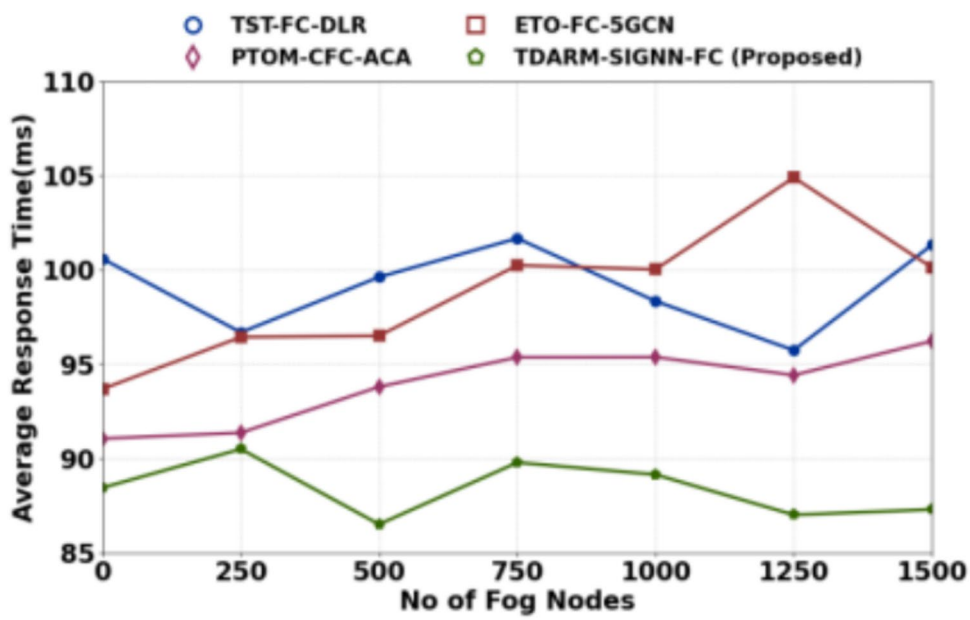

Fig. 7 Performance analysis of average response time

4.1.4 Average response time

The average response time is the time required to offer an initial response to incoming requests across several communication channels. It varies depending on the manner of communication, workload and industry requirements. This is determined using Eq. (34).

$$ART = \frac{Total\,time\,taken\,to\,respond}{Total\,number\,of\,responses} \tag{34}$$

here, ART represents the average response time.

4.1.5 Standard deviation

The square root of a value obtained by comparing data points to the population mean is used to calculate the standard deviation. It is measured by Eq. (35).

$$\text{Standard } Deviation = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \overline{y})^2}{n-1}} \tag{35}$$

where, y_i is signified as value of ith point; $\overline{y}$ indicates the mean value, n is represented as the count of data points.

Table 5 Performance analysis of Standard Deviation (SD)

Method	Standard deviation (dB)					
	Number of fog nodes					
	250	500	750	1000	1250	1500
TST-FC-DLR	100.04	96.34	101.57	104.28	102.92	100.51
PTOM-CFC-ACA	89.45	91.66	94.14	96.65	96.75	95.88
ETO-FC-5GCN	96.63	98.80	98.09	101.83	101.41	106.08
TDARM-SIGNN-FC (Proposed)	87.51	89.91	85.98	89.68	89.60	87.80

Table 6 Performance analysis of cost

Method	Cost ($)					
	Number of fog nodes					
	250	500	750	1000	1250	1500
TST-FC-DLR	90.42	86.88	94.03	97.30	97.56	95.32
PTOM-CFC-ACA	90.06	95.11	97.67	101.58	101.82	101.08
ETO-FC-5GCN	93.63	95.80	95.09	98.83	98.41	103.08
TDARM-SIGNN-FC (Proposed)	88.63	91.73	87.93	92.49	93.53	92.41

Table 7 Ablation study of individual module contributions in the proposed model

Model configuration	Energy consumption (J)	Execution time (ms)	Standard deviation(dB)
Without SFOA (SIGNN only)	112	340	16.8
SIGNN replaced with standard GNN	98	295	13.2
TDARM-SIGNN-FC (Proposed)	85	250	9.5

Table 8 Computational performance of TDARM-SIGNN-FC under different workload scales

Workload scale	Number of tasks	Number of fog nodes	Runtime (s)	Memory usage (MB)	CPU utilization (%)
Small	50	250	2.1	512	35
Medium	200	500	7.8	1024	55
Large	500	1000	21.5	2048	75
Very large	1000	1500	48.7	4096	92

4.1.6 Cost

A cost represents the relationship between several factors that influence the total cost of a project, operation, service, or item. It quantifies the effect of these components' variances on the overall cost. This is calculated using Eq. (36).

$$Total\,\text{Cost} = f\left(y_1, y_2, \ldots\ldots, y_n\right) \tag{36}$$

here, $Total\,\text{Cost}$ denotes the overall cost incurred; $y_1, y_2, \ldots\ldots, y_n$ denotes the input variable and f denotes the function representing the relationship between the input variables and the total cost.

4.1.7 Percentage improvement

Percentage Improvement used to quantify the relative enhancement of a proposed method compared to a baseline or reference method. It is calculated as the reduction or increase in a specific performance metric, expressed as a percentage of the baseline value. This is determined using Eq. (37).

$$Percentage\,\text{Improvement} = \frac{Value\,of\,baseline - Value\,of\,proposed}{Value\,of\,baseline} \times 100\% \tag{37}$$

4.2 Performance analysis

Figure 5, 7 portray the simulation results of TDARM-SIGNN-FC Method. Here, the proposed is TDARM-SIGNN-FC analyzed with other TST-FC-DLR, PTOM-CFC-ACA and ETO-FC-5GCN model respectively.

Figure 5 indicates the proposed model Energy Consumption in achieving the desired results. The SFOA fine-tunes the offloading process reducing unnecessary computations and data transfers and it give decrease energy consumption with best result. The proposed TDARM-SIGNN-FC attains 24.65%, 21.95% and 29.56% lower energy consumption for number of fog nodes at 250; 32.51%, 26.98%, and 31.86% lower energy consumption for count of fog nodes at 500; 25.36%, 21.22% and 19.36% lower energy consumption for count of fog nodes at 750; 32.69%, 24.23% and 22.31% lower energy consumption for count of fog nodes at 1000; 26.35%, 17.23% and 22.33% lower energy consumption for count of fog nodes at 1250 and 18.52%, 25.14% and 30.63% lower energy consumption for count of fog nodes at

1500 compared to the existing TST-FC-DLR, PTOM-CFC-ACA and ETO-FC-5GCN models correspondingly.

Table 4 represents the performance analysis of Total Network Usage. The proposed TDARM-SIGNN-FC consistently demonstrates lower TNU compared to existing approaches, with values ranging from 87.80 to 92.22 bytes, indicating more efficient network utilization. In contrast, TST-FC-DLR, PTOM-CFC-ACA, and ETO-FC-5GCN exhibit higher and more variable TNU values across increasing fog nodes. These results highlight TDARM-SIGNN-FC's ability to minimize network overhead while efficiently managing task distribution, making it highly appropriate for large-scale, real-time fog computing environments.

Figure 6 indicates the proposed model Execution Time in achieving the desired results. The model achieves the lowest execution time due to the efficient task distribution mechanism enabled by SIGNN, which accurately predicts the most suitable fog node for offloading. The proposed TDARM-SIGNN-FC attains 24.78%, 28.97% and 29.45% lower execution time for count of fog nodes at 250; 28.11%, 26.55%, and 31.85% lower execution time for count of fog nodes at 500; 21.66%, 27.65% and 29.30% lower execution time for count of fog nodes at 750; 25.69%, 26.03% and 32.34% lower execution time for count of fog nodes at 1000; 23.36%, 26.20% and 21.63% lower execution time for count of fog nodes at 1250 and 26.53%, 23.33% and 30.30% lower execution time for count of fog nodes at 1500which compared to existing TST-FC-DLR, PTOM-CFC-ACA and ETO-FC-5GCN models respectively.

Figure 7 indicates the proposed model Average Response Time in achieving the desired results. The proposed TDARM-SIGNN-FC model, ART is minimized due to the efficient selection of fog nodes through SIGNN and the SFOA for task offloading. The proposed TDARM-SIGNN-FC attains 26.12%, 32.50% and 19.05% lower average response time for count of fog nodes at 250; 24.45%, 29.12%, and 26.85% lower average response time for count of fog nodes at 500; 31.26%, 17.64% and 19.60% lower average response time for count of fog nodes at 750; 29.64%, 16.53% and 22.44% lower average response time for count of fog nodes at 1000; 26.36%, 29.50% and 23.65% lower average response time for count of fog nodes at 1250 and 21.45%, 24.03% and 20.35% lower average response time for count of fog nodes at 1500 compared to existing TST-FC-DLR, PTOM-CFC-ACA and ETO-FC-5GCN models respectively.

Table 9 Scalability validation of TDARM-SIGNN-FC in large-scale fog computing environments

Fog nodes	Energy consumption (J)	Execution time (ms)	Average response time (ms)	Standard deviation (dB)	Cost ($)
250	85	250	220	9.5	0.60
500	112	278	246	10.8	0.74
750	146	315	289	12.1	0.92
1000	189	362	331	13.9	1.18
1500	225	398	365	15.6	1.42

Table 5 denotes the performance analysis of Standard Deviation. The proposed TDARM-SIGNN-FC consistently achieves lower SD values, ranging from 85.98 dB to 89.91 dB, indicating more stable and reliable performance compared to existing methods. TST-FC-DLR shows higher SD, from 96.34 dB to 104.28 dB, reflecting greater variability in task allocation. PTOM-CFC-ACA and ETO-FC-5GCN also exhibit higher deviations. These results demonstrate that TDARM-SIGNN-FC ensures more consistent resource utilization and task management across fog nodes, improving predictability and overall system stability in large-scale fog computing environments.

Table 6 indicates the performance analysis of cost. The proposed TDARM-SIGNN-FC consistently demonstrates lower operational costs, ranging from 87.93 $ to 93.53 $, compared to TST-FC-DLR with costs between 86.88 $ and 97.56 $, PTOM-CFC-ACA from 90.06 $ to 101.82 $, and ETO-FC-5GCN from 93.08 $ to 103.08 $. These results indicate that TDARM-SIGNN-FC efficiently manages resource allocation while minimizing financial overhead. The lower and more stable costs across increasing fog nodes highlight its effectiveness in large-scale fog computing environments.

4.3 Ablation analysis

This section evaluates the contribution of each component in the proposed model by systematically removing or altering them, demonstrating their individual impact on overall performance and accuracy.

Table 10 Comparative analysis of the proposed and existing approaches

Method	EC (J)	TNU (bytes)	ET (ms)	ART (ms)	SD (dB)	Cost ($)
Choppara, P. and Mangalampalli, S.S., (2024) [25]	166	268	358	354	63.59	3.33
Mangalampalli, S., et.al (2024) [26]	275	254	448	364	52.40	1.52
Muhamad, W.N.W., et.al (2024) [27]	281	362	351	375	31.66	2.36
Umer, A., et.al (2025) [28]	269	282	359	370	20.75	1.25
Wei, Z., et.al (2024) [29]	152	381	363	358	15.82	3.25
Zhang, S.H., et.al (2025) [30]	478	379	458	365	12.52	2.03
Choppara, P. and Mangalampalli, S., (2025) [31]	257	256	374	448	20.86	1.90
TDARM-SIGNN-FC (Proposed)	85	190	250	220	9.5	0.60

Table 7 depicts the ablation study of individual module contributions in the proposed model. The SIGNN alone, without SFOA, results in a greater standard deviation of 16.8 dB, a longer execution time of 340 ms and a higher energy usage of 112 J. Replacing SIGNN with a standard Graph Neural Network improves performance slightly with 98 J, 295 ms, and 13.2 dB, but still lags behind the full TDARM-SIGNN-FC model. The proposed configuration achieves the lowest energy consumption of 85 J, fastest execution time of 250 ms, and minimal deviation of 9.5 dB, demonstrating the effectiveness of combining SIGNN with SFOA.

4.4 Computational complexity and resource analysis

This section evaluates TDARM-SIGNN-FC's computational complexity and resource usage, analyzing runtime, memory, and CPU utilization under varying workloads.

Table 8 depicts the computational performance of TDARM-SIGNN-FC under different workload scales. For a small workload with 50 tasks and 250 fog nodes, the runtime is only 2.1 s with 35% CPU utilization. As the workload increases to 500 tasks and 1000 nodes, runtime rises to 21.5 s and memory usage reaches 2048 MB. In the very large scenario, 1000 tasks require 48.7 s and 92% CPU, showing predictable and manageable scalability.

4.5 Scalability analysis

This section evaluates TDARM-SIGNN-FC's scalability, analyzing performance metrics under increasing fog nodes to demonstrate efficiency and real-time applicability.

Table 9 shows the scalability validation of TDARM-SIGNN-FC in large-scale fog computing environments. When scaling from 250 to 1000 fog nodes, energy consumption rises from 85 to 189 J, while execution time increases from 250 to 362 ms due to higher coordination overhead. Similarly, the average response time grows from 220 to 331 ms, and cost increases from $0.60 to $1.18. Despite this growth, the performance remains stable, validating efficient large-scale fog deployment.

4.6 Comparative analysis

This section provides the comparative analysis of various methods utilized for architectural style highlighting their performance metrics such asEnergy Consumption, Total Network Usage, Execution Time, Average Response Time, Standard Deviation and Cost.

The comparative analysis in the Table 10 highlights the exceptional performance of the TDARM-SIGNN-FC model in comparison to existing methods across key metrics such as Energy Consumption (EC), Total Network Usage (TNU), Execution Time (ET), Average Response Time (ART), Standard Deviation (SD) and Cost. The proposed method consistently outperforms prior techniques, achieving the lowest EC of 85 J, TNU of 190 bytes, ET of 250 ms, ART of 220 ms, SD of 9.5 dB, and Cost of $0.60. These results indicate substantial improvements in efficiency, stability, and resource utilization. In comparison, existing methods show higher energy usage, longer response times, and increased costs. This demonstrates TDARM-SIGNN-FC's effectiveness for real-time, large-scale fog computing applications.

4.7 Discussion

The findings demonstrate that the proposed TDARM-SIGNN-FC technique is enhanced task distribution and adaptive resource management with optimized offloading mechanisms in fog computing is developed in this paper. The proposed TDARM-SIGNN-FC model effectively enhances task distribution and resource management in fog computing by leveraging SIGNN for precise fog node prediction and optimizing task offloading through the SIGNN. But the proposed approach uses a faster SIGNN in combination with the SIGNN algorithm, which makes image collection more effective and improves handling of the model over-fitting issue. The energy consumption values of TDARM-SIGNN-FC is 32.51%, 26.98%, and 31.86% respectively higher than existing methods such as TST-FC-DLR, PTOM-CFC-ACA and ETO-FC-5GCN correspondingly. Analyzed with the precision of current techniques, the proposed method's precision is 98.90%. The proposed approach, TDARM-SIGNN-FC, uses less network overall and evaluates execution times more quickly than other approaches. Thus, compared to the proposed technique, the comparative methods are more costly. As a result, the proposed technique enhanced task distribution and adaptive resource management with optimized offloading mechanisms in fog computing is more efficiently and effectively. Integrating edge AI, neuromorphic computing, blockchain-based job management, and software-defined networking can significantly enhance fog computing. These technologies enable faster, energy-efficient decision-making, secure and transparent task allocation, and flexible, scalable network management, improving overall system performance, reliability, and adaptability in real-time, large-scale edge and fog environments.

5 Conclusion

This research proposed TDARM-SIGNN-FC, a framework for enhanced task distribution and adaptive resource management in fog computing. Using SIGNN for fog node prediction and the SFOA for task offloading, the model efficiently distributes workloads. Experimental results show lower average response time of 220 ms, lower standard deviation of 9.5 dB, lower energy consumption of 85 J, and 0.60 $ cost compared to existing methods. These findings demonstrate improved service quality, reduced latency, and efficient resource utilization in IoT applications at the network edge. The proposed TDARM-SIGNN-FC approach has limitations related to computational complexity, which may increase significantly with very large workloads. Additionally, the convergence characteristics of SFOA can vary depending on task distribution, potentially affecting optimality. Real-world deployment may face challenges such as heterogeneous hardware constraints, dynamic network conditions, and resource-limited edge devices, which could impact performance and scalability. Future research will focus on further enhancing the scalability and adaptability of TDARM-SIGNN-FC in highly dynamic and large-scale fog computing environments. Efforts will include optimizing computational complexity and memory usage to improve efficiency on resource-constrained devices, refining the convergence speed and stability of SFOA, and developing strategies for real-time task allocation and fault-tolerant operation. Additionally, evaluating performance under diverse workload patterns and heterogeneous IoT infrastructures will be considered to ensure broader applicability. And also exploring the impact of edge AI and neuromorphic computing on offloading mechanisms improve predictive analytics and enable proactive resource allocation. The blockchain based decentralized job management mixed with AI-driven trust models has the potential to improve fog network security, transparency and fault tolerance. Implementing software-defined networking (SDN) with AI-powered traffic prediction helps improve dynamic resource provisioning.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10586-026-06025-9.

Acknowledgements None

Author contribution Mrs. Juno Bella Gracia S V—(Corresponding Author)—Conceptualization Methodology, Original draft preparation Dr. Srinivasan S – Supervision.

Funding This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability No datasets were generated or analysed during the current study.

Code availability Not Applicable.

Declarations

Conflicts of interest The authors declare no competing interests.

Consent to participate Not Applicable.

Consent for publication Not Applicable.

Ethical approval This article does not contain any studies with human participants performed by any of the authors.

References

1. Aldossary, M.: Optimizing task offloading for collaborative unmanned aerial vehicles (UAVs) in fog—cloud computing environments. IEEE Access. **12**, 74698 (2024)
2. Fard, A.F., Ardakani, M.M., Mirzaie, K.: Multi-objective task offloading optimization in fog computing environment using INSCSA algorithm. Cluster Comput. **27**(6), 7469–7491 (2024)
3. Sahi, M., Auluck, N., Azim, A., Maruf, M.A.: Dynamic hierarchical intrusion detection task offloading in IoT edge networks. Softw. Pract. Exper. **54**(11), 2249–2271 (2024)
4. Bernard, L., Yassa, S., Alouache, L., Romain, O.: Efficient Pareto based approach for IoT task offloading on Fog-Cloud environments. Internet Things. **27**, 101311 (2024)
5. Khedr, A.M., Alfawaz, O., Alseid, M., El-Moursy, A.: MOTO-MASSA: Multi-objective task offloading based on modified sparrow search algorithm for fog-assisted IoT applications. Wireless Netw. 1–6. (2024)
6. Kumari, N., Jana, P.K.: A metaheuristic-based task offloading scheme with a trade-off between delay and resource utilization in IoT platform. Cluster Comput. **27**(4), 4589–4603 (2024)
7. Lone, K., Sofi, S.A.: e-TOALB: an efficient task offloading in IoT-fog networks. Concur. Comput. Pract. Exp. **36**(6), e7951 (2024)
8. Akyıldız, O., Okay, F.Y., Kök, İ, Özdemir, S.: Mx-TORU: location-aware multi-hop task offloading and resource optimization protocol for connected vehicle networks. Comput. Netw. **259**, 111094 (2025)
9. Nashaat, H., Hashem, W., Rizk, R., Attia, R.: Drl-based distributed task offloading framework in edge-cloud environment. IEEE Access. **12**, 33580 (2024)
10. Gupta, A., Gupta, S.K.: Unmanned aerial vehicles in collaboration with fog computing network for improving quality of service. Int. J. Commun. Syst. **37**(9), e5759 (2024)
11. Moghaddasi, K., Rajabi, S., Gharehchopogh, F.S.: Multi-objective secure task offloading strategy for blockchain-enabled IoV-MEC systems: a double deep Q-network approach. IEEE Access **12**, 3437–3463 (2024)
12. Trabelsi, Z., Ali, M., Qayyum, T.: Fuzzy-based task offloading in Internet of Vehicles (IoV) edge computing for latency-sensitive applications. Internet Things **28**, 101392 (2024)
13. Nematollahi, M., Ghaffari, A., Mirzaei, A.: Task offloading in Internet of Things based on the improved multi-objective aquila optimizer. Signal Image Video Process **18**(1), 545–552 (2024)

14. Mokni, I., Yassa, S.: A multi-objective approach for optimizing IoT applications offloading in fog–cloud environments with NSGA-II. J. Supercomput. **80**(19), 27034–27072 (2024)
15. Baker, T., Al Aghbari, Z., Khedr, A.M., Ahmed, N., Girija, S.: EDITORS: energy-aware dynamic task offloading using deep reinforcement transfer learning in SDN-enabled edge nodes. Internet Things. **25**, 101118 (2024)
16. Bushehrian, O., Moazeni, A.: Deep reinforcement learning-based optimal deployment of IoT machine learning jobs in fog computing architecture. Computing **107**(1), 15 (2025)
17. Lin, H., Yang, L., Guo, H., Cao, J.: Decentralized task offloading in edge computing: an offline-to-online reinforcement learning approach. IEEE Trans. Comput. (2024). https://doi.org/10.1109/TC.2024.3377912
18. Peng, X., Song, S., Zhang, X., Dong, M., Ota, K.: Task offloading for IoAV under extreme weather conditions using dynamic price driven double broad reinforcement learning. IEEE Internet Things J. **11**(10), 17021–17033 (2024)
19. Mashal, H., Rezvani, M.H.: Multiobjective offloading optimization in fog computing using deep reinforcement learning. J. Comput. Netw. Commun. **1**, 6255511 (2024)
20. Liu, Z., Lin, Z., Zhang, Y., Chen, C., Guo, J., Qi, B., Tao, F.: Joint optimization of computational tasks offloading for efficient and secure manufacturing through cloud-fog-edge-terminal architecture. IEEE Transactions Emerg. Topics Comput. Intel. (2024)
21. Shyamlal, S., Prathilothamai, M.: DPAFNet: Brain Tumor Segmentation Using Dual Path Attentive Feature Network. In International Conference on Data Science and Applications (pp. 229-244). Singapore: Springer Nature Singapore. (2024, July).
22. TS, G.: Machine learning for river water quality monitoring: assessing seasonal and agricultural influences. Environmental Earth Sciences, 84(21), 1-30. (2025).
23. Ganesh Babu, R., Geetha T, S., Kavin Kumar, K.: KERN HIC: a hyperspectral remote sensing model for land cover classification and land use monitoring. Environmental Monitoring and Assessment, 197(11), 1275. (2025)
24. Rajesh, M., Sitharthan, R., Ganesh Babu, R., Moorthy, U., Easwaramoorthy, S. V.: Senior Member of IEEE. Spatiotemporal neural radiance fields for AI driven motion quality analysis. Discover Internet of Things, 5(1), 91. (2025).
25. Choppara, P., Mangalampalli, S.S.: a hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning. IEEE Access. **12**, 176363 (2024)
26. Mangalampalli, S., Karri, G.R., Mohanty, S.N., Ali, S., Khan, M.I., Ismail, E., Awwad, F.A.: Prioritized task offloading mechanism in cloud-fog computing using improved asynchronous advantage actor critic algorithm. IEEE Access. **12**, 136628 (2024)
27. Muhamad, W.N., Aris, S.S., Dimyati, K., Javed, M.A., Idris, A., Ali, D.M., Abdullah, E.: Energy-efficient task offloading in fog computing for 5G cellular network. Eng. Sci. Technol. Int. J. **50**, 101628 (2024)
28. Wei, Z., Mao, J., Li, B., Zhang, R.: Privacy-preserving hierarchical reinforcement learning framework for task offloading in low-altitude vehicular fog computing. IEEE Open J. Commun. Soc. **6**, 3389 (2024)
29. Zhang, S.H., Wang, J.S., Zhang, S.W., Xing, Y.X., Sui, X.F.: Multi-strategy fusion snake optimizer on task offloading and scheduling for IoT-based fog computing multi-tasks learning. Cluster Comput. **28**(1), 69 (2025)
30. Choppara, P., Mangalampalli, S.: An efficient deep reinforcement learning based task scheduler in cloud-fog environment. Cluster Comput. **28**(1), 1–26 (2025)
31. Chen, D., Zheng, S., Xu, M., Zhu, Z., Zhao, Y.: SiGNN: a spike-induced graph neural network for dynamic graph representation learning. Pattern Recognit. **158**, 111026 (2025)
32. Zhong, C., Li, G., Meng, Z., Li, H., Yildiz, A.R., Mirjalili, S.: Starfish optimization algorithm (SFOA): a bio-inspired metaheuristic algorithm for global optimization compared with 100 optimizers. Neural Comput. Appl. **37**(5), 3641–3683 (2025)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.